

# **MINOR PROJECT REPORT**

## **STUDENT MANAGEMENT SYSTEM USING PYTHON**

**Submitted By:**

**Thriveni Muli**

Roll No: 232P1A3223

B.Tech

**Submitted To:**

Topgrade Innovation

## Problem Statement

Managing student records manually is difficult, time-consuming, and prone to errors. It becomes hard to store, update, search, and maintain student information efficiently. There is a need for a simple and reliable system that can store student data digitally and perform operations like adding, viewing, searching, and deleting records easily. This project aims to solve these problems by developing a Student Management System using Python.

## Objective

The main objective of this project is to develop a simple and efficient Student Management System using Python. This system is designed to reduce the difficulties of managing student records manually and to improve accuracy and data handling.

The project aims to provide an easy-to-use interface where users can perform basic operations such as adding new student details, viewing all student records, searching for a specific student, and deleting student information when required.

Another important objective is to understand and apply core Python programming concepts such as functions, loops, conditional statements, data structures like lists and dictionaries, and file handling techniques for storing and retrieving data.

The system also focuses on maintaining data consistency and minimizing errors by providing a structured way to handle student information digitally. Overall, this project helps in improving programming skills and demonstrates how Python can be used to build real-world applications

## Tools & Technologies Used

- **Python:** Main programming language
- **IDE (VS Code/IDLE):** Code writing and execution
- **List & Dictionary:** Used to store student data
- **File Handling:** Used to save data permanently
- **JSON:** Used for structured data storage
- **Text File:** *students.txt* for storing records
- **Console Interface:** User interaction

## Algorithm

1. Start program
2. Load data from file
3. Display menu
4. Take user choice
  - Add → Input data, save
  - View → Show all records
  - Search → Find by roll number
  - Delete → Remove record, save
  - Exit → Stop program
5. End

## Implementation Steps

The project is developed using Python with a menu-driven approach. Functions are used to perform operations such as adding, viewing, searching, and deleting student records.

Student data is stored using list and dictionary. File handling is used to store data in a text file using JSON format, ensuring permanent storage.

The system is simple, efficient, and displays output clearly on the console.

# CODE

```
import json

students = []

def load_from_file():
    global students
    try:
        with open("students.txt", "r") as f:
            students = json.load(f)
    except:
        students = []

def save_to_file():
    with open("students.txt", "w") as f:
        json.dump(students, f)

def add_student():
    name = input("Enter name: ")
    roll = input("Enter roll number: ")
    marks = input("Enter marks: ")

    students.append({"name": name, "roll": roll, "marks": marks})
    save_to_file()
    print("Student added successfully!\n")

def view_students():
    if not students:
        print("No records found.\n")
    else:
        for s in students:
            print(s)
    print()

def search_student():
    roll = input("Enter roll number: ")
    for s in students:
        if s["roll"] == roll:
            print("Student Found:", s)
            return
```

```

    print("Student not found.\n")

def delete_student():
    roll = input("Enter roll number: ")
    for s in students:
        if s["roll"] == roll:
            students.remove(s)
            save_to_file()
            print("Student deleted successfully!\n")
            return
    print("Student not found.\n")

# Load existing data
load_from_file()

# Menu-driven program
while True:
    print("1. Add Student")
    print("2. View Students")
    print("3. Search Student")
    print("4. Delete Student")
    print("5. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        add_student()
    elif choice == "2":
        view_students()
    elif choice == "3":
        search_student()
    elif choice == "4":
        delete_student()
    elif choice == "5":
        print("Exiting program...")
        break
    else:
        print("Invalid choice\n")

```

# Output Screenshots

```

PS C:\Users\mulit\OneDrive\TOPGRADE INNOVATIONS\minorproject> & 'c:\Users\mulit\AppData\Local\Python\bin\python.exe' 'c:\Users\mulit\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' 53549' '--' 'c:\Users\mulit\OneDrive\TOPGRADE INNOVATIONS\minorproject\MinorProject_ThriveniMuli.py'
1. Add Student
2. View Students
3. Search Student
4. Delete Student
5. Exit
Enter choice: 1
Enter name: thriveni
Enter roll number: 23
Enter marks: 97
Student added successfully!

1. Add Student
2. View Students
3. Search Student
4. Delete Student
5. Exit
Enter choice: 1
Enter name: bharath
Enter roll number: 32
Enter marks: 98
Student added successfully!

```

## Step 1: Add Student

In this step, new student records are added by entering name, roll number, and marks. The system successfully stores the data.

```

1. Add Student
2. View Students
3. Search Student
4. Delete Student
5. Exit
Enter choice: 2

----- STUDENT RECORDS -----
Name : thriveni
Roll : 23
Marks: 97
-----
Name : bharath
Roll : 32
Marks: 98
-----

```

## Step 2:View Students

The system displays all stored student records. The details are shown correctly from the file.

```

1. Add Student
2. View Students
3. Search Student
4. Delete Student
5. Exit
Enter choice: 3
Enter roll number: 32

Student Found
Name : bharath
Roll : 32
Marks: 98

```

### Step 3: Search Student

The system searches for a student using roll number and displays the correct record if found.

```

1. Add Student
2. View Students
3. Search Student
4. Delete Student
5. Exit
Enter choice: 4
Enter roll number: 23
Student deleted successfully!

1. Add Student
2. View Students
3. Search Student
4. Delete Student
5. Exit
Enter choice: 2

----- STUDENT RECORDS -----
Name : bharath
Roll : 32
Marks: 98
-----

```

### Step 4: Delete Student

After deletion, the updated student list is displayed correctly showing remaining records

```
1. Add Student
2. View Students
3. Search Student
4. Delete Student
5. Exit
Enter choice: 5
Exiting program...
PS C:\Users\mulit\OneDrive\TOPGRADE INNOVATIONS\minorproject>
```

## Step 5 – Exit

The program exits successfully after selecting the exit option. The Student Management System closes properly.

## Result

The Student Management System was successfully developed using Python.

All operations such as adding, viewing, searching, and deleting student records are working correctly.

The system stores data using file handling, ensuring that the data is saved permanently. The program executes without errors and provides accurate results.

## Conclusion

This project helped in understanding important Python concepts such as functions, loops, data structures, and file handling. It also demonstrates how these concepts can be used to develop a real-world application.

The Student Management System makes it easy to manage student data in a simple and efficient way. It reduces manual work and helps in maintaining records properly.

In the future, the system can be improved by adding a graphical user interface (GUI) and connecting it to a database for better performance and usability.

